

NAMES: MUSONI NSHUTI SAM

ID: 28857

COURSE: DATABASE MANAGEMENT SYSTEMS

NOTE: For queries returning multiple columns, "ID : 28857" is concatenated to the **primary** calculated or displayed column as relevant. Elsewhere it's simply concatenated to the output

String Function Exercises (15)

1. Concatenate first and last name as full_name

```
SELECT CONCAT(CONCAT(first_name, ' ', last_name), ' | ID : 28857') AS full_name  
FROM EMPLOYEE;
```

2. Convert all employee names to lowercase

```
SELECT  
CONCAT(LOWER(first_name), ' | ID : 28857') AS first_name_lower,  
CONCAT(LOWER(last_name), ' | ID : 28857') AS last_name_lower  
FROM EMPLOYEE;
```

3. Extract first 3 letters of the employee's first name

```
SELECT CONCAT(LEFT(first_name, 3), ' | ID : 28857') AS first_3_letters  
FROM EMPLOYEE;
```

4. Replace '@company.com' in email with '@org.com'

```
SELECT CONCAT(REPLACE(email, '@company.com', '@org.com'), ' | ID : 28857') AS  
updated_email  
FROM EMPLOYEE;
```

5. Trim spaces from a padded string

```
SELECT CONCAT(TRIM(padded_string), ' | ID : 28857') AS trimmed_string
```

FROM some_table;

6. Count characters in an employee's full name

```
SELECT CONCAT(LENGTH(CONCAT(first_name, ' ', last_name)), ' | ID : 28857') AS
```

```
name_length
```

```
FROM EMPLOYEE;
```

7. Find position of '@' in email

```
SELECT CONCAT(INSTR(email, '@'), ' | ID : 28857') AS at_position
```

```
FROM EMPLOYEE;
```

8. Add 'Mr.' or 'Ms.' before names based on gender

```
SELECT
```

```
CONCAT(
```

```
CASE
```

```
WHEN gender = 'M' THEN CONCAT('Mr. ', first_name, ' ', last_name)
```

```
WHEN gender = 'F' THEN CONCAT('Ms. ', first_name, ' ', last_name)
```

```
ELSE CONCAT(first_name, ' ', last_name)
```

```
END,
```

```
' | ID : 28857'
```

```
) AS titled_name
```

```
FROM EMPLOYEE;
```

9. Format project names to uppercase

```
SELECT CONCAT(UPPER(project_name), ' | ID : 28857') AS project_upper
```

```
FROM PROJECTS;
```

10. Remove any dashes from project names

```
SELECT CONCAT(REPLACE(project_name, '-', ''), ' | ID : 28857') AS project_cleaned
```

FROM PROJECTS;

11. Create a label like “Emp: John Doe (HR)”

SELECT

CONCAT('Emp: ', first_name, ' ', last_name, ' (', d.department_name, ')', ' | ID :
28857') AS label

FROM EMPLOYEE e

JOIN DEPARTMENTS d ON e.department_id = d.department_id;

12. Check email length for each employee

SELECT CONCAT(LENGTH(email), ' | ID : 28857') AS email_length

FROM EMPLOYEE;

13. Extract last name only from email (before @)

SELECT CONCAT(SUBSTRING_INDEX(email, '@', 1), ' | ID : 28857') AS email_username

FROM EMPLOYEE;

14. Format: “LASTNAME, Firstname”

SELECT CONCAT(UPPER(last_name), ', ', first_name, ' | ID : 28857') AS formatted_name

FROM EMPLOYEE;

15. Add “(Active)” next to employee names who have current projects

SELECT

CONCAT(first_name, ' ', last_name,

CASE

WHEN EXISTS (

SELECT 1 FROM EMPLOYEE_PROJECTS ep

JOIN PROJECTS p ON ep.project_id = p.project_id

WHERE ep.employee_id = e.employee_id AND (p.end_date IS NULL OR p.end_date >=

```
CURRENT_DATE)
)
THEN ' (Active)'
ELSE ''
END,
' | ID : 28857'
) AS name_status
FROM EMPLOYEE e;
```

Numeric Function Exercises (10)

16. Round salary to the nearest whole number

```
SELECT employee_id, CONCAT(ROUND(salary), ' | ID : 28857') AS rounded_salary
FROM EMPLOYEE;
```

17. Show only even salaries using MOD

```
SELECT employee_id, CONCAT(salary, ' | ID : 28857') AS salary
FROM EMPLOYEE
WHERE MOD(ROUND(salary), 2) = 0;
```

18. Show difference between two project end/start dates using DATEDIFF

```
SELECT project_id, CONCAT(DATEDIFF(end_date, start_date), ' | ID : 28857') AS
project_duration_days
FROM PROJECTS;
```

19. Show absolute difference in salaries between two employees

```
SELECT CONCAT(ABS(e1.salary - e2.salary), ' | ID : 28857') AS salary_difference
```

FROM EMPLOYEE e1

JOIN EMPLOYEE e2 ON e1.employee_id = 101 AND e2.employee_id = 102;

20. Raise salary by 10% using POWER

SELECT employee_id, salary,

CONCAT(salary * POWER(1.10, 1), ' | ID : 28857') AS increased_salary

FROM EMPLOYEE;

21. Generate a random number for testing IDs

SELECT employee_id, CONCAT(ROUND(RAND() * 10000), ' | ID : 28857') AS test_id

FROM EMPLOYEE;

22. Use CEIL and FLOOR on a floating salary

SELECT employee_id, salary,

CONCAT(CEIL(salary), ' | ID : 28857') AS salary_ceil,

CONCAT(FLOOR(salary), ' | ID : 28857') AS salary_floor

FROM EMPLOYEE;

23. Use LENGTH() on phone numbers

SELECT employee_id, phone_number,

CONCAT(LENGTH(phone_number), ' | ID : 28857') AS phone_length

FROM EMPLOYEE;

24. Categorize salary: High / Medium / Low using CASE

SELECT employee_id, salary,

CONCAT(

CASE

WHEN salary >= 10000 THEN 'High'

WHEN salary >= 5000 THEN 'Medium'

```
ELSE 'Low'

END,

' | ID : 28857'

) AS salary_category

FROM EMPLOYEE;
```

25. Count digits in salary amount

```
SELECT employee_id, salary,

CONCAT(LENGTH(REPLACE(CAST(salary AS CHAR), '.', '')), ' | ID : 28857') AS

digit_count

FROM EMPLOYEE;
```

Date/Time Function Exercises (10)

26. Show today's date using CURRENT_DATE

```
SELECT CONCAT(CURRENT_DATE, ' | ID : 28857') AS today;
```

27. Calculate how many days an employee has worked

```
SELECT employee_id, first_name, last_name,

CONCAT(DATEDIFF(CURRENT_DATE, hire_date), ' | ID : 28857') AS days_worked

FROM EMPLOYEE;
```

28. Show employees hired in the current year

```
SELECT employee_id, first_name, last_name,

CONCAT(hire_date, ' | ID : 28857') AS hire_date

FROM EMPLOYEE

WHERE YEAR(hire_date) = YEAR(CURRENT_DATE);
```

29. Display current date and time using NOW()

```
SELECT CONCAT(NOW(), ' | ID : 28857') AS current_datetime;
```

30. Extract the year, month, and day from hire_date

```
SELECT employee_id,  
  
CONCAT(YEAR(hire_date), ' | ID : 28857') AS hire_year,  
  
CONCAT(MONTH(hire_date), ' | ID : 28857') AS hire_month,  
  
CONCAT(DAY(hire_date), ' | ID : 28857') AS hire_day  
  
FROM EMPLOYEE;
```

31. Show employees hired before 2020

```
SELECT employee_id, first_name, last_name,  
  
CONCAT(hire_date, ' | ID : 28857') AS hire_date  
  
FROM EMPLOYEE  
  
WHERE hire_date < '2020-01-01';
```

32. List projects that ended in the last 30 days

```
SELECT project_id, project_name,  
  
CONCAT(end_date, ' | ID : 28857') AS end_date  
  
FROM PROJECTS  
  
WHERE end_date IS NOT NULL  
  
AND end_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

33. Calculate total days between project start and end dates

```
SELECT project_id, project_name,  
  
CONCAT(DATEDIFF(end_date, start_date), ' | ID : 28857') AS total_days  
  
FROM PROJECTS  
  
WHERE end_date IS NOT NULL;
```

34. Format date '2025-07-23' to 'July 23, 2025' using CONCAT

```
SELECT CONCAT(MONTHNAME('2025-07-23'), ' ', DAY('2025-07-23'), ' ',  
YEAR('2025-07-23'), ' | ID : 28857') AS formatted_date;
```

35. Add a CASE: if project still active (end_date IS NULL), show 'Ongoing'

```
SELECT project_id, project_name,  
CONCAT(  
CASE  
WHEN end_date IS NULL THEN 'Ongoing'  
ELSE 'Completed'  
END,  
' | ID : 28857'  
) AS status  
FROM PROJECTS;
```

Conditional Function Exercises (15)

36. Use CASE to label salaries

```
SELECT employee_id, salary,  
CONCAT(  
CASE  
WHEN salary >= 10000 THEN 'High'  
WHEN salary >= 5000 THEN 'Medium'  
ELSE 'Low'  
END,
```

```
' | ID : 28857'
```

```
) AS salary_label
```

```
FROM EMPLOYEE;
```

37. Use COALESCE to show 'No Email' if email is NULL

```
SELECT employee_id, CONCAT(COALESCE(email, 'No Email'), ' | ID : 28857') AS
```

```
email_display
```

```
FROM EMPLOYEE;
```

38. CASE: If hire_date < 2015, mark as 'Veteran'

```
SELECT employee_id, first_name, hire_date,
```

```
CONCAT(
```

```
CASE
```

```
WHEN hire_date < '2015-01-01' THEN 'Veteran'
```

```
ELSE 'New Hire'
```

```
END,
```

```
' | ID : 28857'
```

```
) AS status
```

```
FROM EMPLOYEE;
```

39. If salary is NULL, default it to 3000 using COALESCE

```
SELECT employee_id, CONCAT(COALESCE(salary, 3000), ' | ID : 28857') AS
```

```
salary_with_default
```

```
FROM EMPLOYEE;
```

40. CASE: Categorize departments (IT, HR, Other)

```
SELECT d.department_id, d.department_name,
```

```
CONCAT(
```

```
CASE
WHEN d.department_name = 'IT' THEN 'IT'
WHEN d.department_name = 'HR' THEN 'HR'
ELSE 'Other'
END,
' | ID : 28857'
) AS department_category
FROM DEPARTMENTS d;
```

41. CASE: If employee has no project, mark as 'Unassigned'

```
SELECT e.employee_id, first_name, last_name,
CONCAT(
CASE
WHEN ep.project_id IS NULL THEN 'Unassigned'
ELSE 'Assigned'
END,
' | ID : 28857'
) AS project_status
FROM EMPLOYEE e
LEFT JOIN EMPLOYEE_PROJECTS ep ON e.employee_id = ep.employee_id;
```

42. CASE: Show tax band based on salary

```
SELECT employee_id, salary,
CONCAT(
CASE
WHEN salary >= 12000 THEN 'Band A'
```

```
WHEN salary >= 8000 THEN 'Band B'
```

```
WHEN salary >= 5000 THEN 'Band C'
```

```
ELSE 'Band D'
```

```
END,
```

```
' | ID : 28857'
```

```
) AS tax_band
```

```
FROM EMPLOYEE;
```

43. Use nested CASE to label project duration

```
SELECT project_id, DATEDIFF(end_date, start_date) AS duration,
```

```
CONCAT(
```

```
CASE
```

```
WHEN end_date IS NULL THEN 'Ongoing'
```

```
WHEN DATEDIFF(end_date, start_date) > 365 THEN 'Long-Term'
```

```
WHEN DATEDIFF(end_date, start_date) > 180 THEN 'Mid-Term'
```

```
ELSE 'Short-Term'
```

```
END,
```

```
' | ID : 28857'
```

```
) AS duration_label
```

```
FROM PROJECTS;
```

44. Use CASE with MOD to show even/odd salary IDs

```
SELECT employee_id, salary,
```

```
CONCAT(
```

```
CASE
```

```
WHEN MOD(employee_id, 2) = 0 THEN 'Even ID'
```

ELSE 'Odd ID'

END,

' | ID : 28857'

) AS id_parity

FROM EMPLOYEE;

45. Combine COALESCE + CONCAT for fallback names

SELECT employee_id,

CONCAT(

COALESCE(first_name, 'Unknown'), ' ', COALESCE(last_name, 'Name'), ' | ID : 28857'

) AS full_name

FROM EMPLOYEE;

46. CASE with LENGTH(): if name length > 10, label “Long Name”

SELECT employee_id, first_name,

CONCAT(

CASE

WHEN LENGTH(first_name) > 10 THEN 'Long Name'

ELSE 'Short Name'

END,

' | ID : 28857'

) AS name_length_category

FROM EMPLOYEE;

47. CASE + UPPER(): if email has ‘TEST’, mark as dummy account

SELECT employee_id, email,

CONCAT(

```
CASE
WHEN UPPER(email) LIKE '%TEST%' THEN 'Dummy Account'
ELSE 'Valid Account'
END,
' | ID : 28857'
) AS account_type
FROM EMPLOYEE;
```

48. CASE: Show seniority based on hire year

```
SELECT employee_id, hire_date,
CONCAT(
CASE
WHEN YEAR(hire_date) <= YEAR(CURRENT_DATE) - 10 THEN 'Senior'
WHEN YEAR(hire_date) <= YEAR(CURRENT_DATE) - 5 THEN 'Mid-Level'
ELSE 'Junior'
END,
' | ID : 28857'
) AS seniority_level
FROM EMPLOYEE;
```

49. Use CASE to determine salary increment range

```
SELECT employee_id, salary,
CONCAT(
CASE
WHEN salary < 3000 THEN 'Increment: 20%'
WHEN salary < 7000 THEN 'Increment: 15%'
```

```
WHEN salary < 10000 THEN 'Increment: 10%'
```

```
ELSE 'Increment: 5%'
```

```
END,
```

```
' | ID : 28857'
```

```
) AS increment_range
```

```
FROM EMPLOYEE;
```

50. Use CASE with CURDATE() to determine anniversary month

```
SELECT employee_id, hire_date,
```

```
CONCAT(
```

```
CASE
```

```
WHEN MONTH(hire_date) = MONTH(CURDATE()) THEN 'Anniversary Month'
```

```
ELSE 'Not This Month'
```

```
END,
```

```
' | ID : 28857'
```

```
) AS anniversary_status
```

```
FROM EMPLOYEE;
```